ELEC3544 — Data Science with Foundation Models
Forecasting JPMorgan Weekly Closing Prices

Bai Yu · Team project

The University of Hong Kong

Abstract

This report studies one-week-ahead weekly closing-price forecasting for JPMorgan Chase (JPM) using public market and macroeconomic information. The empirical objective is not to claim a tradable anomaly, but to test whether a reproducible supervised-learning pipeline can improve on a strong naive persistence baseline (NPB), where the next weekly close is forecast by the current weekly close. The feature set combines stock-specific technical indicators, peer-bank and broad-market returns, gold, Treasury-yield movements, and exchange-rate changes from Yahoo Finance and Federal Reserve Economic Data (FRED). The modelling suite compares ordinary least squares (OLS), ridge, lasso, random forest, and XGBoost using chronological train–test splitting and time-series cross-validation. In the archived replication, lasso obtains the lowest test RMSE among learned models, but the improvement over NPB is only 0.29%; XGBoost underperforms persistence. The main conclusion is therefore conservative: for weekly price levels, disciplined validation and benchmark comparison matter more than model complexity.

Keywords: equity forecasting; JPMorgan Chase; weekly close; FRED; yfinance; lasso; random forest; XGBoost; naive persistence; time-series validation.

1 Introduction

Short-horizon stock-price forecasting is a difficult setting for data science. Publicly traded large-cap bank stocks incorporate information quickly, and weekly prices often behave close to a persistence process. This means that an apparently sophisticated model can look accurate simply because it follows the current price level, while still adding little useful information beyond the last observed close. A serious forecasting report must therefore evaluate learned models against a simple benchmark and must avoid leakage from future observations into preprocessing, tuning, or model selection.

The project addresses this issue through a reproducible pipeline for one-week-ahead prediction of JPMorgan Chase's weekly closing price. The forecast origin is week t and the target is the Friday close in week $t + 1$. All features are measured at or before the close of week t. The resulting task is a supervised regression problem:

$$\hat{y}_{t+1} = f(\mathbf{x}_t),$$

where y_{t+1} is the next weekly close and $\mathbf{x}_t$ is the feature vector available at the forecast date. The project began from a bank-stock forecasting proposal and was implemented as a transparent Python workflow. The present report focuses on the reproducible JPM configuration contained in the code and exported result files.

The study has three research questions. First, do multivariate market and macroeconomic features improve out-of-sample RMSE relative to NPB? Second, among OLS, ridge, lasso, random forest, and XGBoost, which model gives the best trade-off between accuracy, stability, and interpretability? Third, which preprocessing steps are needed to make the comparison credible in a financial time-series setting?

The contribution is practical rather than purely theoretical. The pipeline downloads data, aligns weekly observations, engineers features, screens multicollinearity, performs chronological splitting, fits models, and exports diagnostics in a fixed order. This is valuable because financial forecasting results are easy to overstate when datasets are manually adjusted or when test data influence feature processing. Here, every major modelling decision is designed to preserve the separation between training information and hold-out evaluation.

Two design choices are worth emphasizing at the outset. First, the report evaluates price-level forecasts rather than trading signals. This makes the exercise suitable for comparing regression models and understanding baseline strength, but it does not by itself imply an investment strategy. Second, the project treats reproducibility as part of the empirical result. If another user reruns the pipeline with the same configuration, the expected output is not only a trained estimator but also a frozen set of metrics, figures, and feature lists that can be inspected against the report.

2 Data Sources and Feature Construction

2.1 Raw data and sample window

The dataset combines two public data sources. Yahoo Finance, accessed through `yfinance`, supplies equity and futures price series for JPM, peer banks, a financial-sector ETF, gold futures, and the FTSE 100. FRED supplies macroeconomic and market series including the 10-year Treasury yield, VIX, S&P 500, and USD exchange rates against the euro and sterling. Daily observations are aggregated to week-ending Friday values so that macro and equity variables share a common weekly calendar.

The archived dataset uses 1,737 training weeks and 435 test weeks. The training period runs from 2005-03-11 to 2021-10-29, and the test period runs from 2021-10-30 to 2025-12-27. The chronological 80%/20% split is important: random shuffling would mix regimes and can make validation too optimistic when observations are serially dependent.

The long sample covers several distinct market environments, including the global financial crisis aftermath, the low-rate period of the 2010s, the COVID-19 shock, and the higher-rate period after 2022. This variety is useful because a model that only performs in one quiet regime may not generalize. At the same time, the inclusion of multiple regimes also makes the task harder: coefficients and nonlinear relationships may drift over time, so the hold-out evaluation should be interpreted as a realistic stress test rather than a purely stationary laboratory setting.

2.2 Predictor design

The feature builder creates variables intended to summarize firm-specific momentum, market co-movement, macro conditions, and currency effects. The retained feature set after screening contains fifteen predictors:

`primary_ATR14`, `primary_vol_chg`, `primary_ret_1w`, `primary_Close`, `primary_Volume`, `ratio_yield_gold`, `bac_ret_1w`, `c_ret_1w`, `wfc_ret_1w`, `gspc_ret_1w`, `ftse_ret_1w`, `gold_ret_1w`, `tnx_chg`, `eurusd_ret`, `gbpusd_ret`.

This set includes the current JPM close, the JPM volume and short-horizon technical indicators, the weekly returns of the peer banks, the broad returns of the equity-market, the gold returns, the change in the yield of the Treasury and the movements of the exchange-rate. The current close is intentionally retained because the project forecasts price levels rather than returns, and because the NPB comparison requires the same information timing.

The features can be grouped into four economic channels. The first is stock-specific state, represented by the current close, volume, volatility, and recent return. The second is banking-sector co-movement, represented by Bank of America, Citi, and Wells Fargo returns. The third is broad risk appetite, represented by S&P 500, FTSE, and gold movements. The fourth is macro-financial pressure, represented by Treasury-yield and exchange-rate changes. The design therefore does not rely on one isolated signal; it tests whether a compact multi-source information set adds value beyond persistence.

2.3 Cleaning and leakage controls

The preprocessing workflow applies several controls. Missing values are handled through forward fill, backward fill, time interpolation, median replacement, and finally zero only if a column remains empty. Infinite values are removed or replaced before model fitting. Winsorization cutoffs are estimated using the training sample only and then applied to both train and test data. The current close is exempt from winsorization so that test-period prices are not artificially capped at the training range.

Multicollinearity is handled by iterative variance inflation factor (VIF) screening on the training sample. At each step, the feature with the highest VIF is removed if the maximum exceeds ten, except that `primary_Close` is protected. This produces a more stable regression design while preserving the economic interpretation of a price-level forecast. Finally, standardization is fitted only on the training data, with `primary_Close` left in raw dollar units. The same scaling parameters are reused on the test set.

Figure 1 summarizes the dependence structure of the final training design. The matrix uses the retained predictor names after screening and annotates only larger absolute correlations, so the plot emphasizes the strongest dependence patterns rather than every cell value. This is useful because the project compares linear estimators whose coefficients can become unstable when predictors are strongly correlated. The figure also explains why VIF screening and regularization are reasonable safeguards rather than cosmetic preprocessing steps.

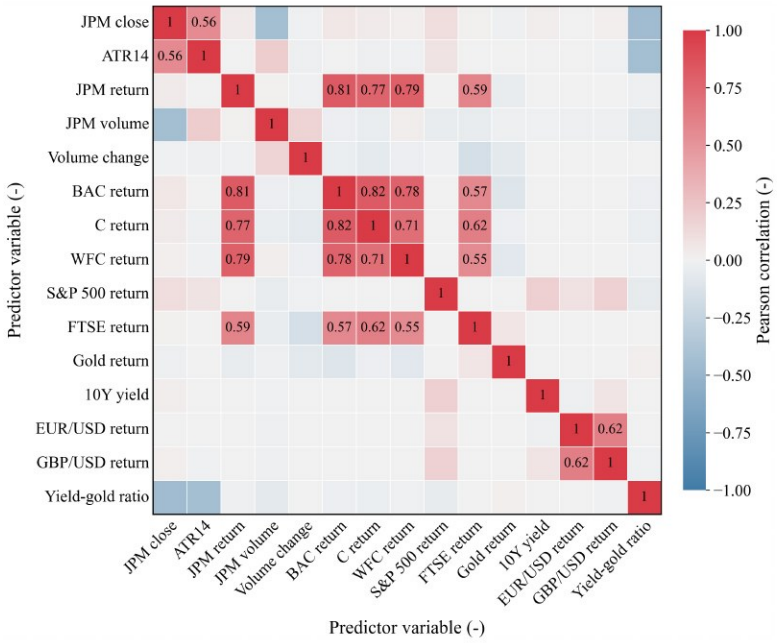


Figure 1: Training-sample feature correlation matrix after preprocessing and feature screening. The color scale shows Pearson correlation, while numeric labels are displayed only for stronger relationships to keep the collinearity structure readable.

3 Methodology

3.1 Benchmark and learned models

The benchmark is the naive persistence baseline:

$$\hat{y}_{t+1}^{\mathrm{NPB}} = y_t.$$

This baseline is deliberately simple but strong. In a weekly price-level task, the most recent close already contains substantial information about the next close. Any machine-learning model should therefore be judged by whether it reduces forecast error relative to this rule, not merely by whether it achieves a high level R^2.

The learned model set contains five estimators. OLS provides an unregularized linear baseline. Ridge regression shrinks coefficients through an ℓ_2 penalty and is useful when predictors are correlated. Lasso uses an ℓ_1 penalty and can set low-contribution coefficients exactly to zero, producing a sparse model that is easier to interpret. Random forest averages randomized decision trees and can capture nonlinearities and interactions. XGBoost builds boosted trees sequentially and can be powerful on tabular datasets, but it is also more flexible and therefore more vulnerable to overfitting in small or noisy samples.

Linear models are trained directly on the next weekly close. The tree models are trained on the residual target

$$r_{t+1} = y_{t+1} - y_t,$$

and final predictions are converted back to levels using

$$\hat{y}_{t+1} = y_t + \hat{r}_{t+1}.$$

This residual formulation is useful because tree models cannot extrapolate smoothly outside the observed range of training labels. Predicting residual changes around the current close reduces the risk that a tree ensemble collapses to stale training-period price levels when the test period reaches new nominal highs.

The models also differ in interpretability. OLS is transparent but can be unstable when predictors are correlated. Ridge is less sparse but usually more stable because it spreads weight across related variables. Lasso is attractive for reporting because its zero coefficients indicate which variables contribute little after controlling for the rest of the design. Random forest and XGBoost can capture threshold effects and interactions, but their flexibility makes them harder to explain and easier to overtune when the number of weekly observations is limited.

3.2 Validation and metrics

The training segment is used for fitting and hyperparameter selection. Ridge, lasso, random forest, and XGBoost are tuned with time-series cross-validation rather than random folds. This respects the ordering of observations and better reflects the information structure of forecasting. The hold-out test set is used only after preprocessing rules and hyperparameters are fixed.

The primary metrics are RMSE, MAE, and level R^2:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \quad \mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|,$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

The report also uses percentage RMSE improvement relative to NPB:

$$\Delta_{\mathrm{NPB}} = \frac{\mathrm{RMSE}_{\mathrm{NPB}} - \mathrm{RMSE}_{\mathrm{model}}}{\mathrm{RMSE}_{\mathrm{NPB}}} \times 100\%.$$

Positive values indicate that a learned model beats persistence; negative values indicate underperformance.

The metric choice affects interpretation. RMSE penalizes large misses more heavily and is therefore useful when large weekly errors are especially undesirable. MAE is more robust to occasional large errors and describes the typical absolute miss. Level R^2 is included for completeness, but it is not the main decision criterion because price-level forecasting with a lagged close naturally produces high explained variation.

4 Results

4.1 Test-set accuracy

Table 1 reports test performance. Lasso achieves the lowest RMSE, at 4.754, which is a 0.29% improvement over NPB. Ridge and OLS are very close, improving RMSE by 0.13% and 0.12%, respectively. Random forest is also close to NPB. XGBoost performs worse than persistence, with RMSE 2.59% higher than the benchmark.

These differences are small in absolute terms. The gap between NPB and lasso is roughly 1.4 cents of RMSE per share in this split, while the test-period weekly price level spans a much larger range. Therefore, the correct reading is not that lasso creates a large predictive edge, but that it offers the cleanest marginal improvement among the tested specifications.

Model	RMSE	MAE	R^2	Δ_{NPB} (%)
NPB	4.768	2.496	0.9941	0.00
OLS	4.762	2.519	0.9941	0.12
Ridge	4.761	2.518	0.9941	0.13
Lasso	4.754	2.549	0.9942	0.29
RF	4.763	2.579	0.9941	0.10
XGBoost	4.891	2.791	0.9938	−2.59

Table 1: Out-of-sample performance on the chronological test partition. RMSE and MAE are in US dollars. Δ_{NPB} measures percentage RMSE gain relative to naive persistence.

The first important observation is that all models have high R^2 values. This should not be overinterpreted. Because the target is a price level and the current close is included as a feature, much of the level variation is mechanically captured by persistence. A high R^2 does not imply profitable return predictability. The more informative comparison is RMSE against NPB, where the gains are small.

The second observation is that model complexity does not guarantee better performance. XGBoost is the most flexible method in the comparison, yet it underperforms. This suggests that the effective signal-to-noise ratio is low and that the test set rewards conservative regularization more than nonlinear flexibility. Lasso performs best because it keeps the model close to persistence while allowing a small number of additional covariates to adjust the forecast.



Figure 2: Actual test-period weekly closes versus NPB. Panel (a) compares the level series, and Panel (b) reports the one-week forecast error. The close overlap in Panel (a) shows the strength of persistence, while Panel (b) identifies the deviations that learned models must reduce.

Figure 2 reinforces the same point visually. Panel (a) shows that the actual close and the NPB forecast nearly overlap over most of the test window, even during the strong rise after 2023 and the drawdown around 2025. Panel (b) isolates the one-week forecast error with positive and negative deviations shown separately. Learned models can only improve meaningfully if the additional features anticipate those deviations; the small RMSE gains suggest that such incremental information is limited in the present feature set.

4.2 Feature interpretation

Figure 3 combines the model-level result with the lasso interpretation. Panel (a) centers all models on NPB, so negative values indicate lower RMSE than the benchmark and positive values indicate underperformance. Panel (b) reports the same comparison as a percentage gain relative to NPB, making the small size of the learned-model improvements visible. Panel (c) shows the sparse nonzero lasso coefficients: the current JPM close dominates, while Treasury-yield and sterling-dollar movements enter as much smaller adjustments.

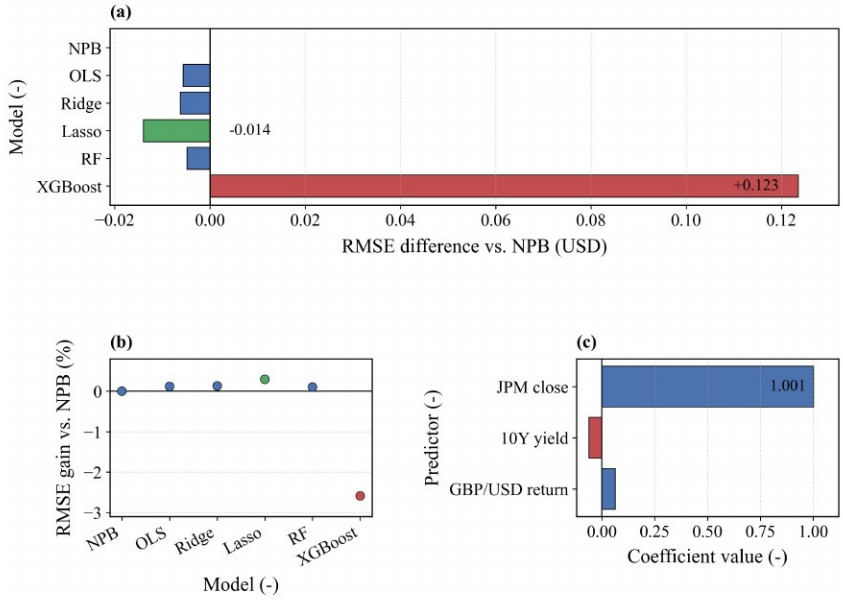


Figure 3: Model performance and lasso interpretation. Panel (a) reports RMSE difference relative to NPB in US dollars, Panel (b) reports RMSE gain relative to NPB in percentage terms, and Panel (c) displays the nonzero lasso coefficients.

The sparse coefficient pattern is useful for interpretation. It shows that lasso does not treat every engineered feature as equally useful. Most peer-bank and broad-market return variables are plausible economic predictors,

but once the current close and other correlated features are present, their incremental contribution to one-week price-level prediction is limited. This is not a failure of feature engineering; rather, it is evidence that a strong benchmark absorbs much of the predictable component at this horizon.

The dominance of `primary_Close` also explains why the project should avoid overstating coefficient interpretation. A variable can be economically relevant for bank valuation but still receive a zero lasso coefficient in this narrow forecasting design. Zero means that the variable did not improve this penalized one-week level forecast after preprocessing and after the current close was included; it does not mean the variable is irrelevant for finance in general.

5 Discussion

The empirical message is cautious. Lasso is the best learned model by RMSE, but its advantage over NPB is only 0.29%. This magnitude is far below the kind of improvement that would support a strong trading claim. If the objective were MAE rather than RMSE, the ranking would also become less favorable to lasso, since NPB has the lowest MAE in Table 1. This illustrates why model selection should be tied to the intended loss function.

The pipeline nevertheless succeeds as a data-science project. It implements a clean sequence from data acquisition to evaluation, avoids lookahead in target construction, estimates preprocessing rules on the training segment only, uses time-ordered validation, and reports a benchmark that is genuinely hard to beat. These design choices make the reported comparison more credible than a model-only demonstration.

From a modelling perspective, the most valuable outcome is the contrast between apparent and incremental accuracy. A report that only showed R^2 would make the models look excellent. Adding NPB changes the conclusion: most of the accuracy comes from the persistence structure of weekly price levels, not from complex learned relationships. This is precisely why benchmark design is central in financial machine learning.

Several limitations remain. First, the analysis forecasts price levels rather than returns. This makes predictions visually accurate but can obscure the economic difficulty of generating actionable excess returns. Second, the study does not include transaction costs, risk limits, position sizing, or portfolio construction. Third, the data are vendor aggregates and may be revised. Fourth, the model is evaluated on one primary stock and one horizon, so results should not be generalized to all banks or all market regimes.

There are also limitations in the feature engineering choices. Imputation can smooth discontinuities around holidays or missing macro releases. Winsorization protects against extreme leverage points, but it may also dampen genuinely informative stress-period observations. VIF screening improves numerical stability, yet it is a heuristic rather than a formal causal selection method. These trade-offs are acceptable for a reproducible coursework pipeline, but a production forecasting system would require additional robustness checks.

Future work could extend the same pipeline in four directions. The target could be changed from price levels to weekly returns, direction, or volatility. The validation scheme could be expanded to rolling or walk-forward testing. The model set could include elastic net and simpler nonlinear learners with stronger regularization. Finally, the project could be rerun with HSBC as the primary ticker if the deliverable must exactly match the initial proposal title; the current repository outputs are internally consistent with JPM.

6 Foundation Models and Project Scope

The forecasting map in this project does not use a pretrained language model or other foundation model to produce predictions. All point forecasts are generated by classical tabular regression models. This is an intentional scope decision. The input data are structured weekly time series, and the main methodological risk is not lack of model expressiveness but leakage, non-stationarity, and overinterpretation of small benchmark gains.

Foundation models can still support the project indirectly. They may help with code review, documentation, literature search, or future text-derived features such as news sentiment. However, any text feature would need the same strict timestamp alignment as the numeric variables. A future extension using news or filings should ensure that each text observation was available before the forecast origin and should compare against the same persistence baseline.

7 Conclusion

This report presents a reproducible one-week-ahead forecasting pipeline for JPM weekly closing prices. The results show that NPB is extremely competitive. Lasso achieves the best RMSE among learned models, but the improvement is modest; XGBoost performs worse than persistence despite its greater flexibility. The key lesson is that in short-horizon financial forecasting, disciplined evaluation is as important as algorithm choice. A simple,

transparent, leakage-aware model can be preferable to a complex learner when the incremental signal is weak.

The final output is therefore not a claim that weekly bank-stock prices are easy to predict. Instead, it is a reproducible empirical study showing how to construct the dataset, protect the test set, compare models fairly, and interpret small improvements with appropriate caution.

References

[1] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.

[2] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[3] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society Series B*, vol. 58, no. 1, pp. 267–288, 1996.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[6] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021. Available: https://otexts.com/fpp3/.

[7] L. Buitinck et al., "API design for machine learning software: Experiences from the scikit-learn project," in *ECML PKDD Workshop*, 2013.

[8] Federal Reserve Bank of St. Louis, FRED economic database. Available: https://fred.stlouisfed.org/.

[9] R. Aroussi et al., `yfinance`: Yahoo Finance market data downloader. Available: https://github.com/ranaroussi/yfinance.